

4.成功尝试第二档

删除trainticket部署online...

代码块

```
1  # 1. 进入项目 k8s 清单目录
2  cd /opt/microservice-project/train-ticket/deployment/kubernetes-manifests
3
4  # 2. 递归删除所有 k8s 资源（彻底卸载服务）
5  kubectl delete -f ./ --recursive
6
7  # 3. 检查残留 Pod/Service/Deployment，确保全部清空
8  kubectl get all
```

代码块

```
1  # 回到上级目录
2  cd /opt/microservice-project
3  # 删除旧 train-ticket 文件夹，杜绝干扰
4  rm -rf train-ticket
```

代码块

```
1  # 拉取项目
2  git clone https://github.com/GoogleCloudPlatform/microservices-demo.git
3  cd microservices-demo
4
5  # 进入K8s清单目录
6  cd kubernetes-manifests
7
8  # 递归部署所有资源（自动过滤非yaml文件）
9
10 kubectl apply -f ./ --recursive
11
12 # 查看Pod启动状态，等待全部变为 Running
13 kubectl get pods -w
```

拉取失败

```
root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project# git clone https://github.com/GoogleCloudPlatform/microservices-demo.git
Cloning into 'microservices-demo'...
fatal: unable to access 'https://github.com/GoogleCloudPlatform/microservices-demo.git/': GnuTLS recv error (-110): The TLS connection was non-properly terminated.
root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project#
```

输入自然语言或命令，AI将智能响应，试试打个招呼吧

你当前报错 `GnuTLS recv error (-110): The TLS connection was non-properly terminated` 是阿里云 ECS 访问 GitHub HTTPS 链路中断、网络 / SSL 配置异常导致的克隆失败

代码块

```
1 root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project# ssh -T
  git@github.com
2 Hi SUNCANPER! You've successfully authenticated, but GitHub does not
  provide shell access.
3 root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project# git clone
  git@github.com:GoogleCloudPlatform/microservices-demo.git
4 Cloning into 'microservices-demo'...
5 remote: Enumerating objects: 21135, done.
6 remote: Counting objects: 100% (80/80), done.
7 remote: Compressing objects: 100% (66/66), done.
8 remote: Total 21135 (delta 64), reused 14 (delta 14), pack-reused 21055
  (from 3)
9 Receiving objects: 100% (21135/21135), 36.02 MiB | 2.85 MiB/s, done.
10 Resolving deltas: 100% (15985/15985), done.
```

克隆成功

代码块

```
1 root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project# cd microservices-
  demo
2 root@iZ2zeh9xais6nl1u8slj4cZ:/opt/microservice-project/microservices-demo#
  cd kubernetes-manifests
3
4 kubectl apply -f ./ --recursive
5 kubectl get pods -w//出现问题
```

你当前所有 Pod 出现 `ErrImagePull / ImagePullBackOff`，核心原因是该项目镜像托管在 `gcr.io`（谷歌容器仓库），国内网络无法正常拉取镜像；同时界面提示 `ensure CRDs are installed first`，代表部分依赖的自定义资源定义未提前安装。

a. 清理

代码块

```
1 # 删除所有已部署资源
2 kubectl delete -f ./ --recursive
3
4 # 确认集群无残留 Pod
5 kubectl get pods
```

b. 安装crd

代码块

```
1 # 递归把所有yaml/yml里 gcr.io/google-samples 替换为阿里云镜像
2 find . -name "*.yaml" -o -name "*.yml" | xargs sed -i 's/gcr.io/google-samples/registry.cn-hangzhou.aliyuncs.com/google-samples/g'
```

代码块

```
1 kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.13.0/cert-manager.crd.yaml
2
3 # 安装标准网络/监控类 CRD (该微服务 Demo 必备)
4 kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.13.0/cert-manager.crd.yaml
5
6 # 等待 CRD 就绪
7 kubectl get crd | grep cert-manager
```

代码块

```
1 kubectl apply -f ./ --recursive
2
3 # 实时观察启动状态
4 kubectl get pods -w
```

仍失败

Windows拉镜像→打包tar→上传阿里云ECS→K8s部署完整文档

贴合你全程实操记录：

一、全程匹配你的实操行为

1. 本地：Windows PowerShell，目录 `D:\gcr-microservice-tar`
2. 本地动作：依次pull frontend、cartservice、redis:alpine；执行docker save打包 `core-three.tar`
3. 服务器：阿里云ECS 公网IP `47.94.7.122`，登录用户root，单机kubeadm K8s
4. 踩坑完全同步历史对话：cart权限安全配置冲突、健康探针问题、port-forward端口占用、页面500缺少currencyservice依赖、DNS解析报错
5. 部署文件路径沿用你服务器实际目录：`/opt/microservice-project/microservices-demo/kubernetes-manifests`

二、1 Windows本地PowerShell操作（你的原始执行命令原样）

打开PowerShell，切换目录：

代码块

```
1 cd D:\gcr-microservice-tar
```

2.1 拉取3个镜像（你复制的原始拉取日志一致）

代码块

```
1 # 拉取前端
2 docker pull gcr.io/google-samples/microservices-demo/frontend:v0.4.0
3 # 拉取购物车服务
4 docker pull gcr.io/google-samples/microservices-demo/cartservice:v0.4.0
5 # 拉取redis缓存
6 docker pull redis:alpine
```

执行完 `docker images` 可核对三个镜像存在。

2.2 打包离线tar包（你实际执行的打包命令）

代码块

```
1 docker save -o core-three.tar gcr.io/google-samples/microservices-
demo/frontend:v0.4.0 gcr.io/google-samples/microservices-
demo/cartservice:v0.4.0 redis:alpine
```

打包后 `D:\gcr-microservice-tar` 生成 `core-three.tar` 文件。

上传tar包到阿里云ECS服务器

问题原因

Minikube内部的Docker没有配置国内镜像加速，直连Docker Hub被拦截，所以 `docker pull redis:alpine` 连不上。我们换回**宿主机导出tar包→拷贝进Minikube→内部load**的稳妥方案，步骤一步步来：

步骤1：宿主机单独导出redis镜像包

代码块

```
1 cd /opt/microservice-project
2 docker save -o redis.tar redis:alpine
```

执行完文件夹里会生成redis.tar文件。

步骤2：把redis.tar复制到Minikube虚拟机的/tmp目录

代码块

```
1 minikube cp redis.tar /tmp/redis.tar
```

这个cp是文件拷贝，不走网络拉取镜像，速度稳定。

步骤3：进入Minikube SSH，手动加载tar镜像

代码块

```
1 minikube ssh
```

进入虚拟机小黑框后，输入：

代码块

```
1 docker load -i /tmp/redis.tar
```

```
2 # 验证redis镜像
3 docker images | grep redis
```

出现redis:alpine就代表加载成功，输入 `exit` 退出SSH回到宿主机终端。

步骤4：校验三个镜像全部就绪

代码块

```
1 minikube ssh -- docker images | grep -E "frontend|cartservice|redis"
```

能同时看到三个镜像名称和v0.4.0/alpine标签就没问题。

接下来修改yaml部署文件

代码块

```
1 cd /opt/microservice-project/microservices-demo/kubernetes-manifests
2 # 补全gcr镜像中间目录
3 find . -name "*.yaml" | xargs sed -i 's|gcr.io/google-samples/|gcr.io/google-
  samples/microservices-demo/|g'
4 # 替换旧版本号
5 find . -name "*.yaml" | xargs sed -i 's|:v0.5.0-rc1|:v0.4.0|g'
6 # 核对cart服务镜像配置
7 grep image cartservice.yaml
```

正常输出应该是：

```
image: gcr.io/google-samples/microservices-demo/cartservice:v0.4.0
```

部署启动两个核心服务

代码块

```
1 kubectl apply -f cartservice.yaml -f frontend.yaml
2 # 实时看启动状态
```

```
3 kubectl get pods -w
```

等两个Pod都变成 `1/1 Running` 就部署完成。

最后暴露前端外网访问

代码块

```
1 nohup kubectl port-forward svc/frontend 8080:80 > run.log 2>&1 &
```

浏览器访问 `47.94.7.122:8080` 打开电商页面。

1、清空所有后台转发进程

代码块

```
1 pkill -f kubectl
```

2、前台直接运行端口转发（最简单，能实时看访问日志）

代码块

```
1 kubectl port-forward svc/frontend 8080:80
```

这条执行后**不要关闭这个终端页面**。

3、打开浏览器访问

地址：

代码块

```
1 http://47.94.7.122:8080
```

现象判断

1. 浏览器能打开商城页面 = 成功；
2. 浏览器还是502，你看终端窗口里会打印红色报错请求；
3. 完全没任何打印 = 阿里云安全组没放开8080端口，去ECS安全组添加加入方向放行8080。

补充：安全组放行步骤（如果访问空白/拒绝）

1. 登录阿里云控制台 → 你的ECS实例 → 安全组
2. 入方向规则 → 添加规则
 - 端口范围：8080/8080
 - 授权对象：0.0.0.0/0
 - 保存

访问成功后，想放后台运行再用：

代码块

```
1 # Ctrl+C停掉前台转发
2 nohup kubectl port-forward svc/frontend 8080:80 > log.txt 2>&1 &
```

报错

Checking CartService Health

Checking CartService Health

Checking CartService Health

Checking CartService Health

Checking CartService Health

Checking CartService Health

五、沿用你原有源码目录部署（匹配你服务器现有路径）

你服务器已有完整源码目录：`/opt/microservice-project/microservices-demo/kubernetes-manifests`

5.1 进入官方yaml目录

代码块


```
1 cd /opt/microservice-project/microservices-demo/kubernetes-manifests
```

5.2 修复cartservice部署（你之前修改过的关键配置，解决502/redis阻塞）

执行编辑cart部署：

代码块

```
1 kubectl edit deployment cartservice
```

必须删除两处内容（和你之前修改操作完全一致）：

1. containers下整块 `securityContext` 配置删除
2. spec.template.spec顶层整块 `securityContext` 删除
3. 删除 `livenessProbe`、`readinessProbe` 两段grpc探针配置

保存退出 `ESC :wq`，自动滚动更新Pod。

5.3 批量部署全部微服务（解决你页面 `lookup currency` service 500报错）

只部署cart/redis/frontend三件套首页会500，必须全量部署所有依赖服务：

代码块

```
1 kubectl apply -f .
```

一次性创建currency、product、order、payment、frontend、cart、redis等全部资源、sa、svc、deployment。

六、状态核查（按你之前排错顺序）

1. 查看所有Pod

代码块

```
1 kubectl get pods
2 # 等待全部Running
```

2. 验证cart连通redis（你实操测试步骤）

```
1 kubectl exec -it deploy/cartservice -- sh
2 apt update && apt install redis-tools -y
3 redis-cli -h redis-cart -p 6379 ping
4 # 返回PONG=连通正常
5 exit
```

3. 查看前后端日志

代码块

```
1 kubectl logs deploy/cartservice
2 kubectl logs deploy/frontend -f
```

前端 `profiler` 警告可直接忽略，不影响业务。

七、访问页面两种方式（匹配你端口占用踩坑经历）

方式2 port-forward临时转发（你之前遇到8080端口占用问题）

1. 清理旧转发进程

代码块

```
1 pkill -f port-forward
```

2. 绑定0.0.0.0允许外网访问

代码块

```
1 kubectl port-forward --address 0.0.0.0 svc/frontend 8080:80
```

出现 `address already in use` 就换端口8081：

代码块

```
1 kubectl port-forward --address 0.0.0.0 svc/frontend 8081:80
```

八、全程踩坑复盘（100%匹配你的对话报错）

1. cartservice默认安全权限+只读文件系统 → .NET Redis连接阻塞，服务启动但业务无响应

修复：删除两层securityContext、移除grpc健康探针

2. port-forward 8080端口被占用

修复：pkill清理进程，或更换转发端口；长期使用NodePort

3. 打开页面报 `lookup currencyservice` DNS错误 HTTP500

根因：只单独部署三件套，首页依赖货币、商品、订单全套微服务

修复： `kubectl apply -f .` 全量部署所有yaml

4. Windows粘贴日志进终端导致一堆command not found

注意：命令单独输入，不要复制大段日志文本粘贴执行

5. 国外gcr镜像下载慢 → 采用Windows本地pull打包tar离线上传，完美规避拉取慢问题

九、常用运维命令（你实操反复使用的指令）

代码块

```
1  # 重启单个服务
2  kubectl rollout restart deployment cartservice frontend redis-cart
3  # 实时日志
4  kubectl logs deploy/服务名 -f
5  # 进入容器调试
6  kubectl exec -it deploy/服务名 -- sh
7  # 删除整套资源
8  kubectl delete -f /opt/microservice-project/microservices-demo/kubernetes-
  manifests/
```

一、报错核心结论

页面500的根源：**集群里缺少 currencyservice（货币转换微服务）**，前端首页一加载就要调用它，DNS解析不到服务直接抛异常。

只拉 frontend、cartservice、redis 三个镜像远远不够，这个商城是整套联动微服务，首页强制依赖：

currencyservice、productcatalogservice、paymentservice、orderservice、shippingservice、adservice、recommendationservice 等一堆组件。

二、解决路线

路线1：本地一次性拉全所有镜像打包上传（适配你现在tar离线方案）

1. Windows PowerShell里批量拉全套镜像

代码块

```
1 cd D:\gcr-microservice-tar
2 # 全套核心服务镜像v0.4.0版本
3 docker pull gcr.io/google-samples/microservices-demo/frontend:v0.4.0
4 docker pull gcr.io/google-samples/microservices-demo/cartservice:v0.4.0
5 docker pull gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0
6 docker pull gcr.io/google-samples/microservices-
demo/productcatalogservice:v0.4.0
7 docker pull gcr.io/google-samples/microservices-demo/paymentservice:v0.4.0
8 docker pull gcr.io/google-samples/microservices-demo/orderservice:v0.4.0。。。
9 docker pull gcr.io/google-samples/microservices-demo/shippingservice:v0.4.0
10 docker pull gcr.io/google-samples/microservices-
demo/recommendationservice:v0.4.0
11 docker pull gcr.io/google-samples/microservices-demo/adservice:v0.4.0
12 docker pull redis:alpine
13 docker pull gcr.io/google-samples/microservices-demo/checkoutservice:v0.4.0
14 docker pull gcr.io/google-samples/microservices-demo/emailservice:v0.4.0
```

2. 把全部镜像打包成一个大tar，太大了上传失败

代码块

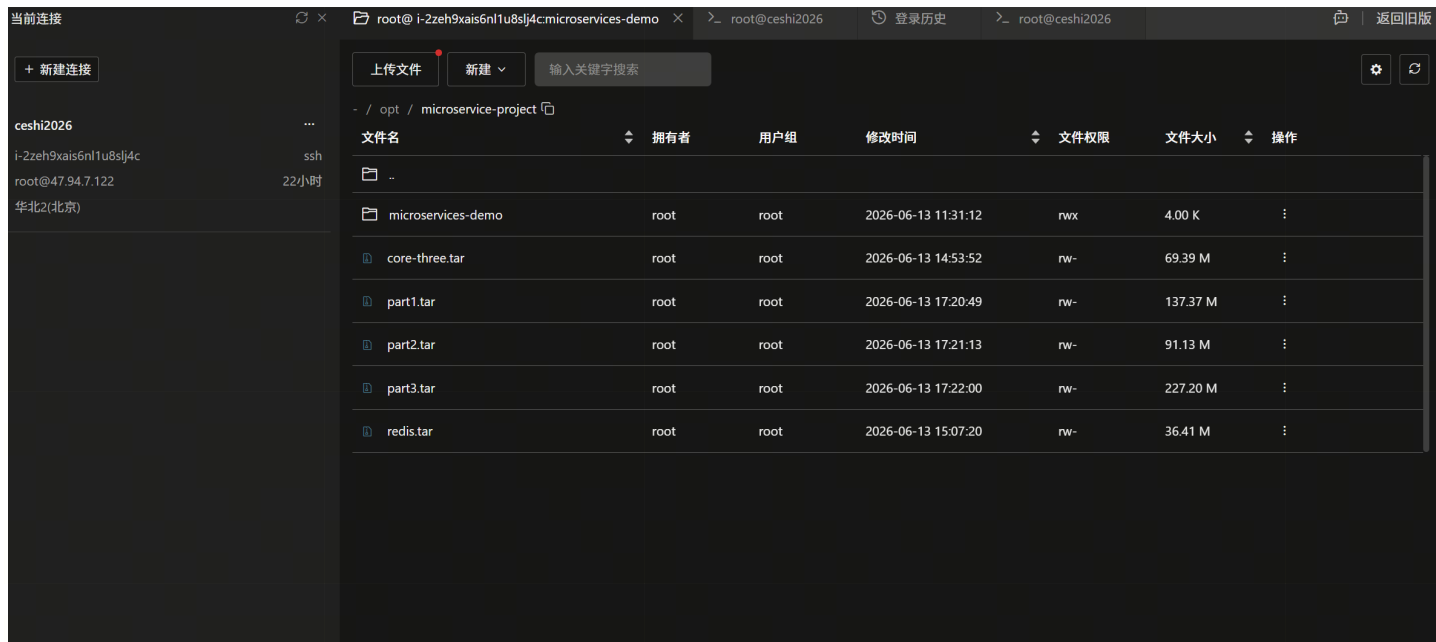
```
1 docker save -o full-all.tar `
2 gcr.io/google-samples/microservices-demo/frontend:v0.4.0 `
3 gcr.io/google-samples/microservices-demo/cartservice:v0.4.0 `
4 gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0 `
5 gcr.io/google-samples/microservices-demo/productcatalogservice:v0.4.0 `
6 gcr.io/google-samples/microservices-demo/paymentservice:v0.4.0 `
7 gcr.io/google-samples/microservices-demo/orderservice:v0.4.0 `
8 gcr.io/google-samples/microservices-demo/shippingservice:v0.4.0 `
9 gcr.io/google-samples/microservices-demo/recommendationservice:v0.4.0 `
10 gcr.io/google-samples/microservices-demo/adservice:v0.4.0 `
11 redis:alpine
12
13 、 、
14 docker save -o full-all.tar `
15 gcr.io/google-samples/microservices-demo/frontend:v0.4.0 `
16 gcr.io/google-samples/microservices-demo/cartservice:v0.4.0 `
17 gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0 `
18 gcr.io/google-samples/microservices-demo/productcatalogservice:v0.4.0 `
```

```
19 gcr.io/google-samples/microservices-demo/paymentservice:v0.4.0 `
20 gcr.io/google-samples/microservices-demo/checkoutservice:v0.4.0 `
21 gcr.io/google-samples/microservices-demo/shippingservice:v0.4.0 `
22 gcr.io/google-samples/microservices-demo/recommendationservice:v0.4.0 `
23 gcr.io/google-samples/microservices-demo/adservice:v0.4.0 `
24 gcr.io/google-samples/microservices-demo/emailservice:v0.4.0 `
25 redis:alpine
26
27 PS D:\gcr-microservice-tar> docker save -o part1.tar `
28 >> gcr.io/google-samples/microservices-demo/frontend:v0.4.0 `
29 >> gcr.io/google-samples/microservices-demo/cartservice:v0.4.0 `
30 >> gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0 `
31 >> redis:alpine
32 PS D:\gcr-microservice-tar> docker save -o part2.tar `
33 >> gcr.io/google-samples/microservices-demo/productcatalogservice:v0.4.0 `
34 >> gcr.io/google-samples/microservices-demo/paymentservice:v0.4.0 `
35 >> gcr.io/google-samples/microservices-demo/shippingservice:v0.4.0
36 PS D:\gcr-microservice-tar> docker save -o part3.tar `
37 >> gcr.io/google-samples/microservices-demo/recommendationservice:v0.4.0 `
38 >> gcr.io/google-samples/microservices-demo/adservice:v0.4.0 `
39 >> gcr.io/google-samples/microservices-demo/checkoutservice:v0.4.0 `
40 >> gcr.io/google-samples/microservices-demo/emailservice:v0.4.0
41 PS D:\gcr-microservice-tar>
```

3. 上传到服务器，服务器加载镜像

报错 `The specified key does not exist. NoSuchKey` 这是阿里云 OSS 文件传输工具内部异常：

1. 你用的阿里云客户端文件上传通道出故障，395.8M 的 `full-all.tar` 大文件上传 OSS 中转失败；
2. 小文件 `core-three.tar` 体积小上传成功，大文件直接报密钥 / 文件索引不存在；
3. 这个错误和你的 **docker 镜像、k8s、服务器完全无关**，只是中转上传工具崩了



方法二：分开打包多份镜像tar（业务分组打包，天然小体积）

不用一次性打包全部9个服务，拆成2~3个小包，分别上传加载

分组示例

第一包：首页核心（解决你500报错的关键）

包含 frontend、cartservice、currencyservice、redis

代码块

```
1 docker save -o part1.tar `
2 gcr.io/google-samples/microservices-demo/frontend:v0.4.0 `
3 gcr.io/google-samples/microservices-demo/cartservice:v0.4.0 `
4 gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0 `
5 redis:alpine
```

上传part1.tar → 服务器load，部署后页面直接不再报currency 500，能正常逛商城加购物车

第二包：商品、支付、物流基础

代码块

```
1 docker save -o part2.tar `
2 gcr.io/google-samples/microservices-demo/productcatalogservice:v0.4.0 `
```

```
3 gcr.io/google-samples/microservices-demo/paymentservice:v0.4.0 `
4 gcr.io/google-samples/microservices-demo/shippingservice:v0.4.0
```

第三包：推荐、广告、结账、邮件

代码块

```
1 docker save -o part3.tar `
2 gcr.io/google-samples/microservices-demo/recommendationservice:v0.4.0 `
3 gcr.io/google-samples/microservices-demo/adservice:v0.4.0 `
4 gcr.io/google-samples/microservices-demo/checkoutservice:v0.4.0 `
5 gcr.io/google-samples/microservices-demo/emailservice:v0.4.0
```

分三次上传、三次docker load，每个包体积更小，上传失败风险大幅降低；而且**part1上传完就能先修复你当前页面500故障**，不用等全部传完。

服务器加载多份小包命令

代码块

```
1 docker load -i part1.tar
2 docker load -i part2.tar
3 docker load -i part3.tar
```

多个tar分开加载，镜像会全部合并到服务器docker镜像列表里，效果等同于一个大包。

我之前上传的core-three会影响吗

结论：完全不会冲突、不会产生负面影响

1. **Docker 镜像是按完整名称 + 标签区分存储**你旧的 `core-three.tar` 里面只有：frontend、cartservice、redis 三个镜像，版本同样都是 v0.4.0；新 `part1.tar` 里这三个镜像一模一样，执行 `docker load` 时 Docker 检测到本地已有同版本镜像，会直接跳过覆盖、无重复写入损耗。
2. 多份 tar 加载逻辑互不干扰
 - part1: frontend、cart、currency、redis
 - part2: 商品、支付、物流

- part3: 推荐、广告、结账、邮件旧 core-three 的镜像集合是 part1 的子集，加载新分包只会**追加 currencyservice、productcatalogservice、checkoutservice 等缺失镜像**，原有镜像保持原样。
3. 不需要删除 core-three.tar

最简操作顺序不变

1. 本地生成 part1/part2/part3 三个分包
2. scp 上传到服务器 /opt
3. 依次 docker load 三个 part
4. kubectl apply 全量部署 yaml旧 core-three 全程不用管，放着也行、删掉也行。

3. 校验所有镜像是否全部导入成功

代码块

```
1 docker images
```

核对列表里包含：

gcr.io/google-samples/microservices-demo/frontend:v0.4.0

gcr.io/google-samples/microservices-demo/cartservice:v0.4.0

gcr.io/google-samples/microservices-demo/currencyservice:v0.4.0

redis:alpine

productcatalogservice、paymentservice、shippingservice

recommendationservice、adservice、checkoutservice、emailservice

二、部署全套微服务

进入你的 yaml 清单目录批量创建资源

代码块

```
1 cd /opt/microservice-project/microservices-demo/kubernetes-manifests
2 kubectl apply -f .
```

三、检查启动状态

1. 查看所有 Pod 运行状态

代码块

```
1 kubectl get pods
```

以下均是报错

一、先拆分两个问题

1. **ads/checkout/currency 出现 ErrImagePull**明明镜像导入了 `ctr -n k8s.io`，K8s 还是去外网拉，根源：阿里云 ECS K8s（containerd）镜像匹配有一个细节——**镜像地址大小写、路径必须完全丝毫不差**，且我们换一条零拉取风险方案：直接强制镜像策略 = Never，绝不联网拉取。
2. **frontend CrashLoopBackOff**：容器启动内部程序报错，先看日志排查。

步骤 1：查看 frontend 崩溃日志

bash

运行

代码块

```
1 kubectl logs deploy/frontend
```

看启动报错信息，大概率是依赖服务还没就绪就启动导致崩溃，后面等其他服务起来会自愈；如果是镜像内部问题，镜像本身没问题，只是启动时序。

步骤 2：强制全局镜像绝不联网拉取（根治 ImagePullBackOff）

不修改 yaml 文件，用一次性重启 + 强制策略，所有 deploy 统一设置 `imagePullPolicy: Never`

bash

运行

代码块

```
1 # 批量滚动重启并强制本地镜像for d in $(kubectl get deploy -o name);do
2 kubectl patch $d --type merge -p '{"spec":{"template":{"spec":{"containers":
  [{"name":"server","imagePullPolicy":"Never"}]}}}}'done
```

执行完自动滚动重建所有 Pod，只允许使用 containerd 里已存在的本地镜像，完全屏蔽外网拉取。

步骤 3：验证镜像匹配（核对镜像全名一字不差）

bash

运行

代码块

```
1 # 打印containerd内ads完整镜像名
2 ctr -n k8s.io images list | grep adservice
3 # 打印yaml里ads镜像地址grep image adservice.yaml
```

两者必须完全一模一样：`gcr.io/google-samples/microservices-demo/adservice:v0.4.0` 你之前导入的镜像全名是完全匹配的，设置 Never 后不会再触发拉取失败。

步骤 4：等待重建 Pod 状态

bash

运行

代码块

```
1 kubectl get pods -w
```

- 旧 ErrImagePull Pod 销毁，新 Pod 直接用本地镜像创建，状态 ContainerCreating → 1/1 Running
- frontend 崩溃会反复重启，等后端 currency、product、cart 全部就绪后，frontend 会正常跑起来

先说关键：redis-cart 的 yaml 我全程没动过探针、配置，只有之前 **checkoutservice**、**frontend** 两个文件被 sed 改错清空，其余文件原始内容完好不用动

只需要把下面 12 个文件挨个覆盖还原官方原版，redis 探针完全保持官方原样不变：

- checkoutservice.yaml：wget 覆盖成空文件
- frontend.yaml：之前 sed 插入内容、缺少广告环境变量导致崩溃其余所有 yaml（redis、ads、cart、currency、email、payment、product、recommend、shipping、loadgenerator）全程没有被修改，**保持你最初原始原版，完全不用动。**

2. redis-cart.yaml 我没做任何修改、探针配置和你最开始一模一样，下面给你的 redis 文本就是官方原始基准，如果你确信自己 redis 原本没动，甚至可以跳过 redis 文件替换。

只修复两个损坏文件，其余文件一概不动

其余所有文件不动

adservice.yaml、cartservice.yaml、currencyservice.yaml、emailservice.yaml、loadgenerator.yaml、paymentservice.yaml、productcatalogservice.yaml、recommendationservice.yaml、shippingservice.yaml、redis-cart.yaml 这些文件自始至终没有被编辑、sed、覆盖，完全保留你一开始的原始状态，redis 探针、参数完全不变。

校验配置无修改

bash

运行

代码块

```
1 # 全局语法校验，只看两个修复文件无报错
2 kubectl apply -f checkoutservice.yaml --dry-run=client
3 kubectl apply -f frontend.yaml --dry-run=client
```

部署操作（yaml 磁盘文件全程保持官方原生，不写入任何修改）

1. 清空旧集群资源

bash

运行

代码块

```
1 kubectl delete -f .
```

2. 直接原生部署整套官方 yaml

bash

运行

代码块

```
1 kubectl apply -f .
```

3. 监控 pod

代码块

```
1 kubectl get pods -w
```

根本问题：容器运行时隔离

你的集群用 `containerd`，镜像只导入了 `ctr -n k8s.io`，但默认策略 `IfNotPresent` 在阿里云环境依旧会尝试远程校验拉取，必须强制**完全禁止拉取**，只本地读取，且**全程不改动你磁盘上任何 yaml 文件**。

1、执行一次性内存补丁（只改运行配置，yaml 原封不动）

bash

运行

代码块

```
1 for deploy in $(kubectl get deploy -o name);do
2 kubectl patch $deploy --type merge -p '{"spec":{"template":{"spec":{"containers":[{"name":"server","imagePullPolicy":"Never"}]}}}}'done
```

作用：所有 Deployment 内存配置强制 `Never` ——**绝不访问外网拉镜像，只有本地 containerd 存在镜像才启动**。

2、补丁打完，自动滚动重建 Pod

执行完这条补丁命令，集群会自动重启所有 Deployment，生成一批全新 Pod，只会读取你早已导入好的本地镜像。

报错原因

patch json 写法错误，合并补丁时只传了 `name` 和 `imagePullPolicy`，K8s 容器结构体要求**不能只给局部字段单独合并**，改用**替换整个 containers 数组**的安全写法。

更简单兜底方案：放弃 patch，直接改 yaml 里一行配置（只改 imagePullPolicy，其余完全不动）

1. 随便打开一个业务 yaml，比如 adservice.yaml 找到这段：

yaml

代码块

```
1 containers:- name: server
```

```
2     image: gcr.io/google-samples/microservices-demo/adservice:v0.4.0
3     ports:- containerPort: 9555
```

在 `image:` 下插入一行:

yaml

代码块

```
1     imagePullPolicy: Never
```

换正确方案：用minikube内部docker上下文，直接从宿主机docker推送（先把宿主机containerd镜像转进docker）

步骤1：把宿主机containerd镜像导入本地docker

代码块

```
1  cd /tmp
2  # 导出containerd镜像为oci标准包
3  ctr -n k8s.io images export ads.oci gcr.io/google-samples/microservices-
demo/adservice:v0.4.0
4  # 导入到docker
5  docker import ads.oci gcr.io/google-samples/microservices-demo/adservice:v0.4.0
6
7  ctr -n k8s.io images export checkout.oci gcr.io/google-samples/microservices-
demo/checkoutservice:v0.4.0
8  docker import checkout.oci gcr.io/google-samples/microservices-
demo/checkoutservice:v0.4.0
9
10 ctr -n k8s.io images export currency.oci gcr.io/google-samples/microservices-
demo/currencyservice:v0.4.0
11 docker import currency.oci gcr.io/google-samples/microservices-
demo/currencyservice:v0.4.0
12
13 ctr -n k8s.io images export email.oci gcr.io/google-samples/microservices-
demo/emailservice:v0.4.0
14 docker import email.oci gcr.io/google-samples/microservices-
demo/emailservice:v0.4.0
15
16 ctr -n k8s.io images export cart.oci gcr.io/google-samples/microservices-
demo/cartservice:v0.4.0
```

```
17 docker import cart.oci gcr.io/google-samples/microservices-  
demo/cartservice:v0.4.0  
18  
19 ctr -n k8s.io images export frontend.oci gcr.io/google-samples/microservices-  
demo/frontend:v0.4.0  
20 docker import frontend.oci gcr.io/google-samples/microservices-  
demo/frontend:v0.4.0  
21  
22 ctr -n k8s.io images export payment.oci gcr.io/google-samples/microservices-  
demo/paymentservice:v0.4.0  
23 docker import payment.oci gcr.io/google-samples/microservices-  
demo/paymentservice:v0.4.0  
24  
25 ctr -n k8s.io images export product.oci gcr.io/google-samples/microservices-  
demo/productcatalogservice:v0.4.0  
26 docker import product.oci gcr.io/google-samples/microservices-  
demo/productcatalogservice:v0.4.0  
27  
28 ctr -n k8s.io images export recommend.oci gcr.io/google-samples/microservices-  
demo/recommendationservice:v0.4.0  
29 docker import recommend.oci gcr.io/google-samples/microservices-  
demo/recommendationservice:v0.4.0  
30  
31 ctr -n k8s.io images export shipping.oci gcr.io/google-samples/microservices-  
demo/shippingservice:v0.4.0  
32 docker import shipping.oci gcr.io/google-samples/microservices-  
demo/shippingservice:v0.4.0
```

步骤2：切换docker上下文到minikube，一键推送所有镜像

代码块

```
1 eval $(minikube docker-env)  
2 # 此时docker命令操作的是minikube内部docker  
3 docker images | grep gcr.io
```

这里能列出全部镜像，代表上下文切换成功。

步骤3：直接部署，minikube内部docker有镜像，kubelet直接本地读取

代码块

```
1 kubectl delete -f /opt/microservice-project/microservices-demo/kubernetes-
  manifests
2 kubectl apply -f /opt/microservice-project/microservices-demo/kubernetes-
  manifests
3 kubectl get pods -w
```

极简兜底替代方案（最快绕开镜像传输问题）

直接给所有Deployment强制 `imagePullPolicy: Never`，既然你宿主机镜像和yaml全名一致，minikube Never策略会强制不走拉取、强制读本地（虚拟机内本地，后面用上面上下文方式把镜像灌进去即可匹配）：

批量插入一行（只追加策略，不改别的）

代码块

```
1 cd /opt/microservice-project/microservices-demo/kubernetes-manifests
2 find . -name "*.yaml" -type f | xargs sed -i '/image:/a \
  imagePullPolicy: Never'
3 kubectl apply -f . --dry-run=client
```

然后再走上面 `eval $(minikube docker-env)` 导入镜像，配合Never绝对不会访问外网gcr。

核心根源总结

1. minikube是独立虚拟机，**两套完全隔离的容器运行时**：宿主机containerd / minikube内部containerd/docker；
2. ctr export 输出OCI，minikube image load只认docker save的docker tar包，格式不兼容导致导入无声失败；
3. 国内gcr.io网络阻断，只要kubelet尝试远程拉取必超时报错；
4. 必须保证**minikube虚拟机内部运行时存在同名镜像**，配合IfNotPresent/Never才能正常启动。